

JSON, API REST y Fetch

Cómo JavaScript intercambia datos con servicios web



ESTUDIANTE	BOOTCAMP	FECHA
Juan Andrés Suárez Gaviria	Bootcamp en Desarrollo Web Fullstack	2 de septiembre de 2026

¿Qué es JSON?

DEFINICIÓN

JSON significa JavaScript Object Notation. Es un formato de texto utilizado para representar e intercambiar datos. Aunque su sintaxis se parece a los objetos de JavaScript, JSON es independiente del lenguaje y puede ser usado por muchas tecnologías.

Estructura y tipos permitidos

EJEMPLO DE DOCUMENTO JSON

```
1 {  
2   "nombre": "Laura",  
3   "edad": 24,  
4   "activo": true,  
5   "habilidades": ["HTML", "CSS", "JS"],  
6   "direccion": {  
7     "ciudad": "Medellín"  
8   },  
9   "telefono": null  
10 }
```

Tipo	Ejemplo
String	"Laura"
Number	24
Boolean	true / false
Null	null
Object	{ "clave": "valor" }
Array	["HTML", "CSS"]

Reglas básicas

Regla	Descripción
Claves	Siempre deben escribirse entre comillas dobles.
Strings	También utilizan comillas dobles.
Separación	Cada propiedad se separa con coma, excepto la última.
No permitido	No admite comentarios, funciones ni undefined.

Convertir entre JSON y JavaScript

OBJETO A JSON

```
1 const usuario = { nombre: "Ana", edad: 22 };  
2 const textoJSON = JSON.stringify(usuario);  
3  
4 console.log(textoJSON);  
5 // {"nombre":"Ana","edad":22}
```

JSON A OBJETO

```
1 const texto =  
2   '{"producto":"Mouse","precio":85000}';  
3 const producto = JSON.parse(texto);  
4  
5 console.log(producto.precio); // 85000
```

Idea clave: JSON es texto. **JSON.stringify()** crea ese texto y **JSON.parse()** lo convierte nuevamente en un valor de JavaScript.

¿Qué es una API REST?

DEFINICIÓN

Una API REST es una interfaz que permite que dos aplicaciones se comuniquen usando las reglas de HTTP. Organiza la información como recursos, los identifica mediante URL y normalmente intercambia sus representaciones en formato JSON.

Cómo funciona



Anatomía de un endpoint

https://api.ejemplo.com/usuarios/15

URL BASE
api.ejemplo.com

RECURSO
/usuarios

IDENTIFICADOR
/15

Métodos HTTP y códigos de estado

Método	Acción habitual
GET	Consultar uno o varios recursos.
POST	Crear un recurso nuevo.
PUT	Reemplazar completamente un recurso.
PATCH	Actualizar una parte del recurso.
DELETE	Eliminar un recurso.

Código	Significado
200	Solicitud correcta
201	Recurso creado
204	Correcto sin contenido
400	Solicitud incorrecta
404	Recurso no encontrado
500	Error del servidor

Características principales

RECURSOS

SIN ESTADO

MÉTODOS HTTP

DATOS EN JSON

¿Qué hace fetch en JavaScript?

DEFINICIÓN

fetch() es una función de la API Fetch disponible para JavaScript en navegadores y entornos modernos. Recibe una URL, realiza una solicitud HTTP y devuelve una Promise que se resuelve con un objeto **Response**. Después, el cuerpo de la respuesta puede leerse como JSON, texto u otro formato.

Flujo de una solicitud

1 **fetch(url)**

2 **Promise**

3 **Response**

4 **.json()**

5 **Datos JS**

Sintaxis básica

SOLICITUD GET CON PROMESAS

```
1 fetch("https://api.ejemplo.com/datos")
2   .then((response) => {
3     if (!response.ok) {
4       throw new Error(`Error HTTP: ${response.status}`);
5     }
6     return response.json();
7   })
8   .then((datos) => console.log(datos))
9   .catch((error) => console.error(error));
```

Elementos importantes de la respuesta

Elemento	Para qué sirve
response.ok	Indica si el código HTTP se encuentra entre 200 y 299.
response.status	Contiene el código recibido, por ejemplo 200 o 404.
response.headers	Permite consultar los encabezados de la respuesta.
response.json()	Lee el cuerpo y devuelve otra Promise con datos de JavaScript.
response.text()	Lee el cuerpo como una cadena de texto.

Importante: **fetch()** rechaza la Promise ante errores de red, pero una respuesta HTTP 404 o 500 normalmente sigue llegando como Response. Por eso se debe comprobar **response.ok**.

Ejemplos con solicitudes GET

Ejemplo 1: obtener un usuario con .then()

GET CON PROMESAS

```
1 fetch("https://jsonplaceholder.typicode.com/users/1")
2   .then((response) => {
3     if (!response.ok) {
4       throw new Error(`Error HTTP: ${response.status}`);
5     }
6     return response.json();
7   })
8   .then((usuario) => {
9     console.log(usuario.name);
10    console.log(usuario.email);
11  })
12  .catch((error) => console.error(error));
```

La primera llamada a `.then()` recibe la respuesta HTTP. `response.json()` procesa el cuerpo y el siguiente `.then()` recibe el objeto del usuario.

Ejemplo 2: recorrer usuarios con async y await

GET CON ASYNC/AWAIT

```
1 async function cargarUsuarios() {
2   try {
3     const respuesta = await fetch(
4       "https://jsonplaceholder.typicode.com/users"
5     );
6
7     if (!respuesta.ok) {
8       throw new Error(`Error HTTP: ${respuesta.status}`);
9     }
10
11     const usuarios = await respuesta.json();
12     usuarios.forEach((usuario) => {
13       console.log(`${usuario.id}: ${usuario.name}`);
14     });
15   } catch (error) {
16     console.error("No fue posible cargar los usuarios", error);
17   }
18 }
19
20 cargarUsuarios();
```

`async/await` ofrece otra forma de trabajar con Promises. `await` espera cada resultado dentro de la función y `try/catch` controla los errores.

Ejemplos con POST y el DOM

Ejemplo 3: enviar datos con POST

CREAR UNA PUBLICACIÓN

```
1 async function crearPublicacion() {
2   const nuevaPublicacion = {
3     title: "Mi primera publicación",
4     body: "Contenido enviado desde JavaScript",
5     userId: 1
6   };
7
8   try {
9     const respuesta = await fetch(
10      "https://jsonplaceholder.typicode.com/posts",
11      {
12        method: "POST",
13        headers: { "Content-Type": "application/json" },
14        body: JSON.stringify(nuevaPublicacion)
15      }
16    );
17    if (!respuesta.ok) throw new Error(respuesta.status);
18    const resultado = await respuesta.json();
19    console.log(resultado);
20  } catch (error) {
21    console.error("No fue posible crear el recurso", error);
22  }
23 }
```

La opción `method` define POST, el encabezado anuncia JSON y `JSON.stringify()` convierte el objeto antes de enviarlo.

Ejemplo 4: mostrar usuarios en una página

HTML

```
1 <h2>Usuarios</h2>
2 <ul id="listaUsuarios"></ul>
```

JAVASCRIPT

```
1 const lista = document.querySelector("#listaUsuarios");
2
3 fetch("https://jsonplaceholder.typicode.com/users")
4   .then((response) => {
5     if (!response.ok) throw new Error(response.status);
6     return response.json();
7   })
8   .then((usuarios) => {
9     usuarios.slice(0, 3).forEach((usuario) => {
10      const item = document.createElement("li");
11      item.textContent =
12        `${usuario.name} - ${usuario.email}`;
13      lista.appendChild(item);
14    });
15  })
16  .catch((error) => console.error(error));
```

Este ejemplo obtiene datos desde una API de prueba, toma los tres primeros usuarios y crea un elemento `<li>` para mostrar cada nombre y correo en el navegador.

Proceso aplicado en los ejemplos

1. SOLICITAR

`fetch` envía la petición.

2. VALIDAR

`response.ok` revisa el estado.

3. CONVERTIR

`json()` procesa el cuerpo.

4. UTILIZAR

Los datos se muestran o guardan.